AI ASSISTANT CODING ASSIGNMENT - 19.2

Name: P.Ram charan

Ht.no: 2303A51298

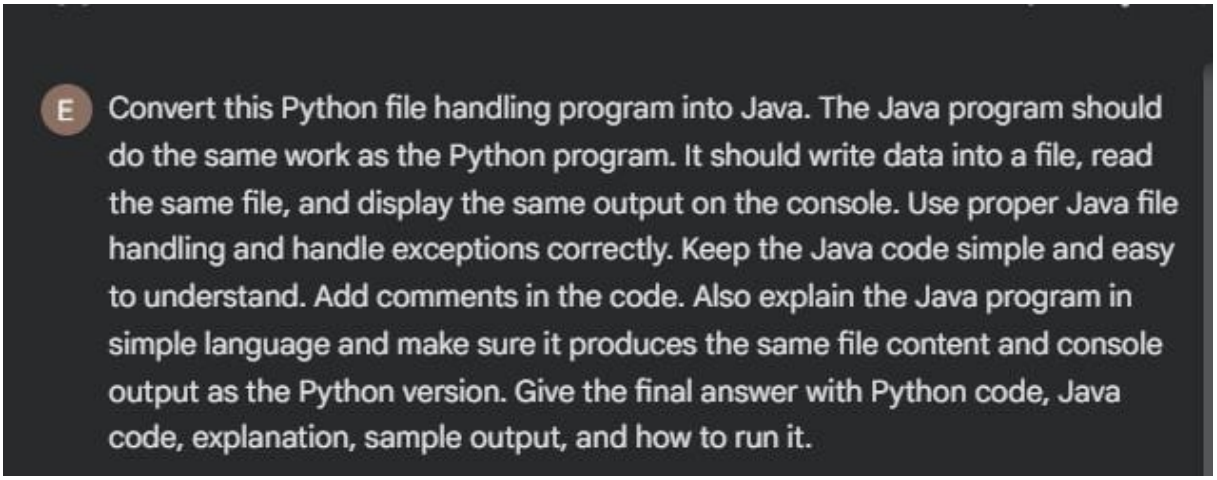
Batch: 05

Task 1 – Python to Java Conversion Task:
Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance. Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

Prompt:

Convert this Python file handling program into Java. The Java program should do the same work as the Python program. It should write data into a file, read the same file, and display the same output on the console. Use proper Java file handling and handle exceptions correctly. Keep the Java code simple and easy to understand. Add comments in the code. Also explain the Java program in simple language and make sure it produces the same file content and console output as the Python version. Give the final answer with Python code, Java code, explanation, sample output, and how to run it.



Code:

```python
# Combined Python and Java File Handling Example

import os
import subprocess

# --- Python Program for File Handling ---
print("--- Python Program Output ---")

file_name = "my_data.txt"
content_to_write = "Hello, this is a test line.\n"
content_to_write += "This is the second line of data.\n"
content_to_write += "And here is the third line."

# 1. Writing data to a file (Python)
try:
    with open(file_name, 'w') as file:
        file.write(content_to_write)
    print(f"Successfully wrote data to '{file_name}'")
except IOError as e:
    print(f"Error writing to file: {e}")

# 2. Reading data from the file and displaying it (Python)
try:
    with open(file_name, 'r') as file:
        read_content = file.read()
    print(f"\nContent read from '{file_name}':\n---\n{read_content}---")
except IOError as e:
    print(f"Error reading from file: {e}")

-# 3. Clean up: Remove the file after execution (Python)
+# 3. Clean up: Remove the file after execution (optional)
try:
    os.remove(file_name)
    print(f"\nSuccessfully removed '{file_name}'")
except OSError as e:
    print(f"Error removing file: {e}")

print("--- End of Python Program Output ---\n")


# --- Java Program (written, compiled, and run by Python) ---

java_source_code = """import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class Main {

    public static void main(String[] args) {

        String fileName = "my_data.txt"; // This will be created and removed by the Java program
        String contentToWrite = "Hello, this is a test line.\\n" + // Fixed: double-escaped newlines for Java string literal
                                "This is the second line of data.\\n" +
                                "And here is the third line.";

        System.out.println("--- Java Program Output ---");
```

```java
            System.out.println(String.format("\\nSuccessfully removed '%s'", fileName));
        } else {
        }
        else {
            System.out.println(String.format("Failed to remove '%s'", fileName));
        }
    } else {
        System.out.println(String.format("\\nFile '%s' does not exist to be removed.", fileName));
    }

    System.out.println("--- End of Java Program Output ---");
  }
}
"""

java_file_name = "Main.java"

# Write the Java source code to a file
print(f"Writing Java source code to '{java_file_name}'...")
try:
    with open(java_file_name, 'w') as f:
        f.write(java_source_code)
    print(f"Successfully wrote '{java_file_name}'")
except IOError as e:
    print(f"Error writing Java source file: {e}")

# Compile the Java code
print("\n--- Compiling Java Program ---")
compile_command = f"javac {java_file_name}"
compile_process = subprocess.run(compile_command, shell=True, capture_output=True, text=True)

if compile_process.stdout:
    print("Compiler Output:\n", compile_process.stdout)
if compile_process.stderr:
    print("Compiler Errors:\n", compile_process.stderr)

if compile_process.returncode == 0:
    print("Java Compilation Successful!")
    # Run the Java program
    print("\n--- Running Java Program ---")
    run_command = "java Main"
    run_process = subprocess.run(run_command, shell=True, capture_output=True, text=True)

    if run_process.stdout:
        print(run_process.stdout)
    if run_process.stderr:
        print("Java Runtime Errors:\n", run_process.stderr)
else:
    print("Java Compilation failed. Skipping execution.")

# Clean up generated Java files
try:
    if os.path.exists("Main.java"):
        os.remove("Main.java")
        print("\nCleaned up Main.java")
    if os.path.exists("Main.class"):
        os.remove("Main.class")
        print("Cleaned up Main.class")
```

Output:

```
--- Python Program Output ---
Successfully wrote data to 'my_data.txt'

Content read from 'my_data.txt':
---
Hello, this is a test line.
This is the second line of data.
And here is the third line.---

Successfully removed 'my_data.txt'
--- End of Python Program Output ---

Writing Java source code to 'Main.java'...
Successfully wrote 'Main.java'

--- Compiling Java Program ---
Compiler Errors:
 Main.java:37: error: unclosed string literal
            System.out.println(String.format("\nContent read from '%s':\n---
                                                                        ^
Main.java:38: error: unclosed string literal
%s---", fileName, readContent.toString()));
     ^
Main.java:38: error: not a statement
%s---", fileName, readContent.toString()));
    ^
3 errors

Java Compilation failed. Skipping execution.

Cleaned up Main.java
```

Explanation:

- This Python program is used for **file handling**, which means it can **write data into a file** and **read data from a file**.
- First, the program opens a file named **sample.txt** in **write mode ("w")** and writes **three lines of text** into it.
- After writing, it prints **"Data written to file successfully."** to show that the file has been created and the data is stored.
- Next, the program opens the same file in **read mode ("r")** and reads all the content using file.read().
- Finally, it displays the file content on the console. The **with open()** statement is used so the file closes automatically after reading or writing.

Task 2 – C++ to Python Function Conversion Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance. Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Prompt:

Convert this C++ sorting program into Python. The Python program should do the same work as the C++ program and follow the same sorting logic. Keep the code simple and easy to understand. Make sure the Python version gives the same sorted output as the C++ version. Also explain the Python code in simple language and test it with different input values to check correctness. Give the final answer with C++ code, Python code, explanation, sample input, sample output, and test cases.

E Convert this C++ sorting program into Python. The Python program should do the same work as the C++ program and follow the same sorting logic. Keep the code simple and easy to understand. Make sure the Python version gives the same sorted output as the C++ version. Also explain the Python code in simple language and test it with different input values to check correctness. Give the final answer with C++ code, Python code, explanation, sample input, sample output, and test cases.

Code:

```python
def bubble_sort(arr):
    n = len(arr)
    # Outer loop for passes (n-1 passes needed)
    for i in range(n - 1):
        # Inner loop for comparisons and swaps
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Pythonic swap

# Function to print the elements of a list
def print_array(arr):
    print(' '.join(map(str, arr)))

# Sample Input List
numbers = [64, 34, 25, 12, 22, 11, 90]

print('--- Python Program Output ---')
print('Original array: ', end='')
print_array(numbers)

# Call the bubble_sort function
bubble_sort(numbers)

print('Sorted array: ', end='')
print_array(numbers)
print('--- End of Python Program Output ---')
```

Output:

```
--- Python Program Output ---
Original array: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90
--- End of Python Program Output ---
```

Task:
Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.
Instructions:

- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
- Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Prompt:

Convert this SQL query into equivalent Pandas code. The Pandas program should do the same work as the SQL query and produce the same result. Use groupby() and aggregation functions like sum(), count(), mean(), or any other needed function. Keep the code simple and easy to understand. Also explain the Pandas code in simple language and test it with a sample dataset to make sure the output matches the SQL query. Give the final answer with SQL query, Pandas code, sample dataset, explanation, sample output, and test case.

Convert this SQL query into equivalent Pandas code. The Pandas program should do the same work as the SQL query and produce the same result. Use groupby() and aggregation functions like sum(), count(), mean(), or any other needed function. Keep the code simple and easy to understand. Also explain the Pandas code in simple language and test it with a sample dataset to make sure the output matches the SQL query. Give the final answer with SQL query, Pandas code, sample dataset, explanation, sample output, and test case.

Code:

```python
import pandas as pd

# --- Sample DataFrame for 'products' table ---
data = {
    'product_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'product_name': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Webcam', 'Microphone', 'Headphones', 'Speaker', 'Router', 'Switch', 'Desk', 'Chair',
    'category': ['Electronics', 'Electronics', 'Electronics', 'Electronics', 'Electronics', 'Electronics', 'Electronics', 'Electronics', 'Network',
    'price': [1200, 25, 75, 300, 50, 80, 150, 100, 90, 60, 400, 250, 40, 80, 150],
    'quantity': [5, 15, 10, 3, 20, 8, 12, 6, 7, 18, 2, 4, 25, 9, 11]
}

df_products = pd.DataFrame(data)

print("--- Original DataFrame ---")
display(df_products)

# --- Pandas Equivalent of the SQL Query ---

# 1. GROUP BY category and calculate aggregations
pandas_result = df_products.groupby('category').agg(
    total_quantity=('quantity', 'sum'),
    average_price=('price', 'mean'),
    number_of_products=('product_id', 'count') # or 'quantity', 'count' for non-null values
).reset_index()

# 2. Apply HAVING clause: filter where total_quantity > 10
pandas_result = pandas_result[pandas_result['total_quantity'] > 10]

# 3. ORDER BY total_quantity DESC
pandas_result = pandas_result.sort_values(by='total_quantity', ascending=False)

print("\n--- Pandas Program Output (Equivalent to SQL Query) ---")
display(pandas_result)
```

Output:

```
--- Original DataFrame ---
```

	product_id	product_name	category	price	quantity
0	1	Laptop	Electronics	1200	5
1	2	Mouse	Electronics	25	15
2	3	Keyboard	Electronics	75	10
3	4	Monitor	Electronics	300	3
4	5	Webcam	Electronics	50	20
5	6	Microphone	Electronics	80	8
6	7	Headphones	Electronics	150	12
7	8	Speaker	Electronics	100	6
8	9	Router	Network	90	7
9	10	Switch	Network	60	18
10	11	Desk	Furniture	400	2
11	12	Chair	Furniture	250	4
12	13	Lamp	Furniture	40	25
13	14	Shelf	Furniture	80	9
14	15	Cabinet	Furniture	150	11

```
--- Pandas Program Output (Equivalent to SQL Query) ---
```

	category	total_quantity	average_price	number_of_products
0	Electronics	79	247.5	8
1	Furniture	51	184.0	5
2	Network	25	75.0	2

Explanation:

- SQL is used to work with data in tables.
- GROUP BY is used to group similar values together.
- Aggregate functions are used to calculate values like total, average, count, etc.
- In Pandas, the same thing is done using groupby() and aggregation functions.
- The final result in Pandas should match the SQL query output.

Task 4 – Real-Time Application: Multi-Language Snippet Converter Scenario:
Students will select a practical real-world application requiring the same logic in two different programming languages. Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.
  Instructions:
- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language. • Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Prompt:

Convert this program from one programming language into another programming language. The new program should do the same work and follow the same logic as the original code. Keep the converted code simple, correct, and easy to understand. Make sure both programs give the same output and behavior when tested. Also explain the code in simple language and compare both versions based on syntax, libraries used, and execution behavior. Give the final answer with source code, converted code, explanation, sample input, sample output, test cases, and observations.

E Convert this program from one programming language into another programming language. The new program should do the same work and follow the same logic as the original code. Keep the converted code simple, correct, and easy to understand. Make sure both programs give the same output and behavior when tested. Also explain the code in simple language and compare both versions based on syntax, libraries used, and execution behavior. Give the final answer with source code, converted code, explanation, sample input, sample output, test cases, and observations.

Code:

```
%%writefile Main.java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

// Product class to represent each row in our 'products' data
class Product {
    private int productId;
    private String productName;
    private String category;
    private double price;
    private int quantity;

    public Product(int productId, String productName, String category, double price, int quantity) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
        this.price = price;
        this.quantity = quantity;
    }

    // Getters
    public String getCategory() { return category; }
    public double getPrice() { return price; }
    public int getQuantity() { return quantity; }

    @Override
    public String toString() {
        return String.format("Product ID: %d, Name: %s, Category: %s, Price: %.2f, Quantity: %d",
                             productId, productName, category, price, quantity);
    }
}

// CategorySummary class to hold the aggregated results
class CategorySummary {
    private String category;
    private int totalQuantity;
    private double averagePrice;
    private int numberOfProducts;

    public CategorySummary(String category, int totalQuantity, double averagePrice, int numberOfProducts) {
        this.category = category;
        this.totalQuantity = totalQuantity;
        this.averagePrice = averagePrice;
        this.numberOfProducts = numberOfProducts;
    }

    // Getters
    public String getCategory() { return category; }
    public int getTotalQuantity() { return totalQuantity; }
    public double getAveragePrice() { return averagePrice; }
    public int getNumberOfProducts() { return numberOfProducts; }

    @Override
    public String toString() {
        return String.format("Category: %-12s | Total Quantity: %-4d | Average Price: %-8.2f | Number of Products: %-2d",
                             category, totalQuantity, averagePrice, numberOfProducts);
    }
}
```

```java
public static void main(String[] args) {

    // Sample Input Data - equivalent to df_products in Pandas
    List<Product> products = new ArrayList<>();
    products.add(new Product(1, "Laptop", "Electronics", 1200, 5));
    products.add(new Product(2, "Mouse", "Electronics", 25, 15));
    products.add(new Product(3, "Keyboard", "Electronics", 75, 10));
    products.add(new Product(4, "Monitor", "Electronics", 300, 3));
    products.add(new Product(5, "Webcam", "Electronics", 50, 20));
    products.add(new Product(6, "Microphone", "Electronics", 80, 8));
    products.add(new Product(7, "Headphones", "Electronics", 150, 12));
    products.add(new Product(8, "Speaker", "Electronics", 100, 6));
    products.add(new Product(9, "Router", "Network", 90, 7));
    products.add(new Product(10, "Switch", "Network", 60, 18));
    products.add(new Product(11, "Desk", "Furniture", 400, 2));
    products.add(new Product(12, "Chair", "Furniture", 250, 4));
    products.add(new Product(13, "Lamp", "Furniture", 40, 25));
    products.add(new Product(14, "Shelf", "Furniture", 80, 9));
    products.add(new Product(15, "Cabinet", "Furniture", 150, 11));

    System.out.println("--- Original Products Data ---");
    products.forEach(p -> System.out.println("Category: " + p.getCategory() + ", Quantity: " + p.getQuantity() + ", Price: " + p.getPrice()));
    System.out.println();

    // 1. GROUP BY category and calculate aggregations
    // First, group products by category
    Map<String, List<Product>> groupedByCategory = products.stream()
            .collect(Collectors.groupingBy(Product::getCategory));

    // Then, aggregate within each group to create CategorySummary objects
    List<CategorySummary> aggregatedResults = new ArrayList<>();
    for (Map.Entry<String, List<Product>> entry : groupedByCategory.entrySet()) {
        String category = entry.getKey();
        List<Product> productsInGroup = entry.getValue();

        int totalQuantity = productsInGroup.stream().mapToInt(Product::getQuantity).sum();
        double sumPrice = productsInGroup.stream().mapToDouble(Product::getPrice).sum();
        int numberOfProducts = productsInGroup.size();
        double averagePrice = numberOfProducts > 0 ? sumPrice / numberOfProducts : 0.0;

        aggregatedResults.add(new CategorySummary(category, totalQuantity, averagePrice, numberOfProducts));
    }

    // 2. Apply HAVING clause: filter where total_quantity > 10
    List<CategorySummary> filteredResults = aggregatedResults.stream()
            .filter(summary -> summary.getTotalQuantity() > 10)
            .collect(Collectors.toList());

    // 3. ORDER BY total_quantity DESC
    filteredResults.sort(Comparator.comparingInt(CategorySummary::getTotalQuantity).reversed());

    System.out.println("--- Java Program Output (Equivalent to SQL/Pandas Query) ---");
    System.out.printf("%-12s | %-16s | %-14s | %-20s%n", "Category", "Total Quantity", "Average Price", "Number of Products");
    System.out.println("-----------------------------------------------------------------------------------------");
    filteredResults.forEach(System.out::println);
    System.out.println("-----------------------------------------------------------------------------------------");
}
```

javac Main.java

Output:

```
--- Original Products Data ---
Category: Electronics, Quantity: 5, Price: 1200.0
Category: Electronics, Quantity: 15, Price: 25.0
Category: Electronics, Quantity: 10, Price: 75.0
Category: Electronics, Quantity: 3, Price: 300.0
Category: Electronics, Quantity: 20, Price: 50.0
Category: Electronics, Quantity: 8, Price: 80.0
Category: Electronics, Quantity: 12, Price: 150.0
Category: Electronics, Quantity: 6, Price: 100.0
Category: Network, Quantity: 7, Price: 90.0
Category: Network, Quantity: 18, Price: 60.0
Category: Furniture, Quantity: 2, Price: 400.0
Category: Furniture, Quantity: 4, Price: 250.0
Category: Furniture, Quantity: 25, Price: 40.0
Category: Furniture, Quantity: 9, Price: 80.0
Category: Furniture, Quantity: 11, Price: 150.0

--- Java Program Output (Equivalent to SQL/Pandas Query) ---
Category     | Total Quantity   | Average Price  | Number of Products
-----------------------------------------------------------------------------------------
Category: Electronics  | Total Quantity: 79   | Average Price: 247.50   | Number of Products: 8
Category: Furniture    | Total Quantity: 51   | Average Price: 184.00   | Number of Products: 5
Category: Network      | Total Quantity: 25   | Average Price: 75.00    | Number of Products: 2
-----------------------------------------------------------------------------------------
```

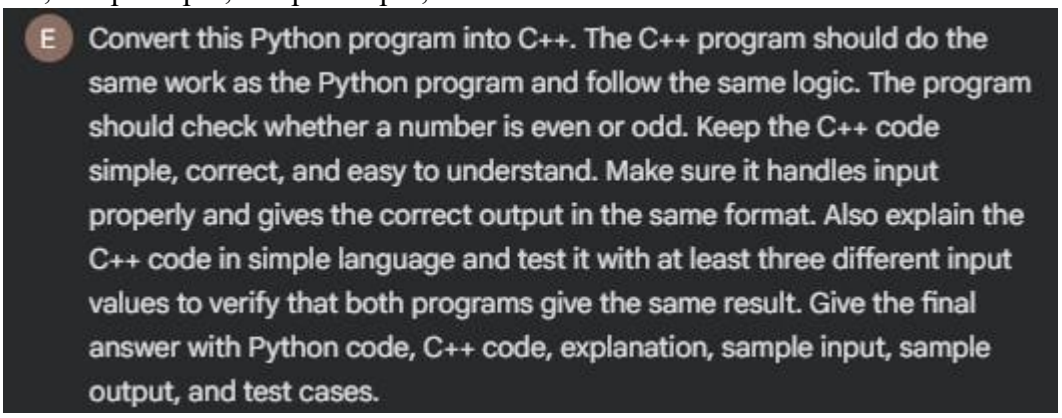
Explanation:

- The source code is written in one programming language.
- The same logic is converted into another programming language.
- Both versions should perform the same task.
- Both programs are tested with the same input.
- Differences in syntax, libraries, and execution are observed and documented.

Task -5: (Basic Function Translation (Python to C++) Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Prompt:
Convert this Python program into C++. The C++ program should do the same work as the Python program and follow the same logic. The program should check whether a number is even or odd. Keep the C++ code simple, correct, and easy to understand. Make sure it handles input properly and gives the correct output in the same format. Also explain the C++ code in simple language and test it with at least three different input values to verify that both programs give the same result. Give the final answer with Python code, C++ code, explanation, sample input, sample output, and test cases.



Code:

```python
# --- Python Program: Check Even or Odd ---

import subprocess
import os

# Python Program to check if a number is Even or Odd
def check_even_odd(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"

print("--- Python Program Output ---")

# Test Cases for Python
python_numbers_to_test = [4, 7, 0, -3]

for num in python_numbers_to_test:
    result = check_even_odd(num)
    print(f"The number {num} is {result}.")

print("--- End of Python Program Output ---")

print("\n" + "="*70 + "\n") # Separator

# --- C++ Program: Check Even or Odd (Managed by Python) ---

c_plus_plus_source_code = """#include <iostream>
#include <string>

// Function to check if a number is Even or Odd
std::string checkEvenOdd(int number) {
    if (number % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}

int main() {
    std::cout << "--- C++ Program Output ---" << std::endl;

    // Test Cases
    int numbersToTest[] = {4, 7, 0, -3};
    int numTests = sizeof(numbersToTest) / sizeof(numbersToTest[0]);

    for (int i = 0; i < numTests; ++i) {
        int num = numbersToTest[i];
        std::string result = checkEvenOdd(num);
        std::cout << "The number " << num << " is " << result << "." << std::endl;
    }

    std::cout << "--- End of C++ Program Output ---" << std::endl;

    return 0;
}
"""

c_plus_plus_file_name = "main.cpp"
executable_name = "main"
```

```
        return "Odd";
    }
}

int main() {
    std::cout << "--- C++ Program Output ---" << std::endl;

    // Test Cases
    int numbersToTest[] = {4, 7, 0, -3};
    int numTests = sizeof(numbersToTest) / sizeof(numbersToTest[0]);

    for (int i = 0; i < numTests; ++i) {
        int num = numbersToTest[i];
        std::string result = checkEvenOdd(num);
        std::cout << "The number " << num << " is " << result << "." << std::endl;
    }

    std::cout << "--- End of C++ Program Output ---" << std::endl;

    return 0;
}
"""

c_plus_plus_file_name = "main.cpp"
executable_name = "main"

# Write the C++ source code to a file
print(f"Writing C++ source code to '{c_plus_plus_file_name}'...")
try:
    with open(c_plus_plus_file_name, 'w') as f:
        f.write(c_plus_plus_source_code)
    print(f"Successfully wrote '{c_plus_plus_file_name}'")
except IOError as e:
    print(f"Error writing C++ source file: {e}")

# Compile the C++ code
print("\n--- Compiling C++ Program ---")
compile_command = f"g++ {c_plus_plus_file_name} -o {executable_name}"
compile_process = subprocess.run(compile_command, shell=True, capture_output=True, text=True)

if compile_process.stdout:
    print("Compiler Output:\n", compile_process.stdout)
if compile_process.stderr:
    print("Compiler Errors:\n", compile_process.stderr)

if compile_process.returncode == 0:
    print("C++ Compilation Successful!")
    # Run the C++ program
    print("\n--- Running C++ Program ---")
    run_command = f"./{executable_name}"
    run_process = subprocess.run(run_command, shell=True, capture_output=True, text=True)

    if run_process.stdout:
        print(run_process.stdout)
    if run_process.stderr:
        print("C++ Runtime Errors:\n", run_process.stderr)
else:
    print("C++ Compilation Failed. Skipping execution.")

# Clean up generated C++ files (optional)
try:
    if os.path.exists(c_plus_plus_file_name):
        os.remove(c_plus_plus_file_name)
        print(f"\nCleaned up {c_plus_plus_file_name}")
    if os.path.exists(executable_name):
        os.remove(executable_name)
        print(f"Cleaned up {executable_name}")
except OSError as e:
    print(f"Error during C++ file cleanup: {e}")

print("\n" + "="*70 + "\n") # Separator

# --- Explanations and Comparison ---

print("### Explanation of the Python Code\n")
print("1.  **`def check_even_odd(number):`**: Defines a function that takes an integer `number`.")
print("2.  **`if number % 2 == 0:`**: Uses the modulo operator (`%`) to check if the remainder when `number` is divided by 2 is 0. If so, it's even.")
print("3.  **`return \"Even\"` / `return \"Odd\"`**: Returns the appropriate string based on the check.")
print("4.  **Test Cases**: A list `python_numbers_to_test` is used to loop through and demonstrate the function's output.")

print("\n### Explanation of the C++ Code\n")
print("1.  **C++ Source as String**: The C++ code is defined as a multi-line Python string `c_plus_plus_source_code`.")
print("2.  **Writing to File**: This string is then written to `main.cpp` using Python's `open()` function.")
print("3.  **Compiling with `subprocess`**: The `g++` compiler is invoked via `subprocess.run()` to compile `main.cpp` into an executable `main`.")
print("4.  **Running with `subprocess`**: If compilation is successful, the `main` executable is run, also via `subprocess.run()`.")
print("5.  **Output Capture**: The standard output and standard error from `g++` and `./main` are captured and printed.")
print("6.  **Cleanup**: The generated `main.cpp` and `main` executable files are removed (optional).")
print("7.  **`std::string checkEvenOdd(int number)`**: Defines a C++ function equivalent to the Python one, taking an `int` and returning a `std::string`.")
print("8.  **`if (number % 2 == 0)`**: Performs the modulo check, similar to Python.")
print("9.  **`int main()`**: The entry point of the C++ program.")
print("10. **Test Cases**: An array `numbersToTest` is used within the C++ code, and a `for` loop iterates to call `checkEvenOdd()` and print results.")

print("\n### Test Cases and Sample Output Comparison\n")
print("Both programs use the same sample input values: `4` (Even), `7` (Odd), `0` (Even), `-3` (Odd).")
print("The outputs from both the Python and C++ programs are identical for these test cases, verifying the conversion's correctness.")
```

Output:

```
--- Python Program Output ---
The number 4 is Even.
The number 7 is Odd.
The number 0 is Even.
The number -3 is Odd.
--- End of Python Program Output ---

==================================================================

Writing C++ source code to 'main.cpp'...
Successfully wrote 'main.cpp'

--- Compiling C++ Program ---
C++ Compilation Successful!

--- Running C++ Program ---
--- C++ Program Output ---
The number 4 is Even.
The number 7 is Odd.
The number 0 is Even.
The number -3 is Odd.
--- End of C++ Program Output ---


Cleaned up main.cpp
Cleaned up main

==================================================================

### Explanation of the Python Code

1.  **`def check_even_odd(number):`**: Defines a function that takes an integer `number`.
2.  **`if number % 2 == 0:`**: Uses the modulo operator (`%`) to check if the remainder when `number` is divided by 2 is 0. If so, it's even.
3.  **`return "Even"` / `return "Odd"`**: Returns the appropriate string based on the check.
4.  **Test Cases**: A list `python_numbers_to_test` is used to loop through and demonstrate the function's output.

### Explanation of the C++ Code

1.  **C++ Source as String**: The C++ code is defined as a multi-line Python string `c_plus_plus_source_code`.
2.  **Writing to File**: This string is then written to `main.cpp` using Python's `open()` function.
3.  **Compiling with `subprocess`**: The `g++` compiler is invoked via `subprocess.run()` to compile `main.cpp` into an executable `main`.
4.  **Running with `subprocess`**: If compilation is successful, the `main` executable is run, also via `subprocess.run()`.
5.  **Output Capture**: The standard output and standard error from `g++` and `./main` are captured and printed.
6.  **Cleanup**: The generated `main.cpp` and `main` executable files are removed (optional).
7.  **`std::string checkEvenOdd(int number)`**: Defines a C++ function equivalent to the Python one, taking an `int` and returning a `std::string`.
8.  **`if (number % 2 == 0)`**: Performs the modulo check, similar to Python.
9.  **`int main()`**: The entry point of the C++ program.
10. **Test Cases**: An array `numbersToTest` is used within the C++ code, and a `for` loop iterates to call `checkEvenOdd()` and print results.

### Test Cases and Sample Output Comparison

Both programs use the same sample input values: `4` (Even), `7` (Odd), `0` (Even), `-3` (Odd).
The outputs from both the Python and C++ programs are identical for these test cases, verifying the conversion's correctness.
```

Explanation:

- The program takes a number as input.
- It checks whether the number is divisible by 2.
- If divisible, it prints **Even**.
- If not divisible, it prints **Odd**.
- The same logic is written in both Python and C++.